

```

/*
 * Display Device Information
 *
 * Script to print out some information about the OpenCL devices
 * and platforms available on your system
 *
 * History: C++ version written by Tom Deakin, 2012
 *          Ported to C by Tom Deakin, July 2013
 *          Updated by Tom Deakin, October 2014
 */

#include <stdio.h>
#include <stdlib.h>

#ifdef __APPLE__
#include <OpenCL/opencl.h>
#include <unistd.h>
#else
#include <CL/cl.h>
#endif

#include "err_code.h"

int main(void)
{
    cl_int err;
    // Find the number of OpenCL platforms
    cl_uint num_platforms;
    err = clGetPlatformIDs(0, NULL, &num_platforms);
    checkError(err, "Finding platforms");
    if (num_platforms == 0)
    {
        printf("Found 0 platforms!\n");
        return EXIT_FAILURE;
    }
    // Create a list of platform IDs
    cl_platform_id platform[num_platforms];
    err = clGetPlatformIDs(num_platforms, platform, NULL);
    checkError(err, "Getting platforms");

    printf("\nNumber of OpenCL platforms: %d\n", num_platforms);
    printf("\n-----\n");

    // Investigate each platform
    for (int i = 0; i < num_platforms; i++)
    {
        cl_char string[10240] = {0};
        // Print out the platform name
        err = clGetPlatformInfo(platform[i], CL_PLATFORM_NAME, sizeof(string), &string, NULL);
        checkError(err, "Getting platform name");
        printf("Platform: %s\n", string);

        // Print out the platform vendor
        err = clGetPlatformInfo(platform[i], CL_PLATFORM_VENDOR, sizeof(string), &string, NULL);
        checkError(err, "Getting platform vendor");
        printf("Vendor: %s\n", string);

        // Print out the platform OpenCL version
        err = clGetPlatformInfo(platform[i], CL_PLATFORM_VERSION, sizeof(string), &string, NULL);
        checkError(err, "Getting platform OpenCL version");
        printf("Version: %s\n", string);

        // Count the number of devices in the platform
        cl_uint num_devices;
        err = clGetDeviceIDs(platform[i], CL_DEVICE_TYPE_ALL, 0, NULL, &num_devices);
        checkError(err, "Finding devices");

        // Get the device IDs
        cl_device_id device[num_devices];

```

```

err = clGetDeviceIDs(platform[i], CL_DEVICE_TYPE_ALL, num_devices, device, N
ULL);
checkError(err, "Getting devices");
printf("Number of devices: %d\n", num_devices);

// Investigate each device
for (int j = 0; j < num_devices; j++)
{
    printf("\t-----\n");

    // Get device name
    err = clGetDeviceInfo(device[j], CL_DEVICE_NAME, sizeof(string), &string
, NULL);
    checkError(err, "Getting device name");
    printf("\t\tName: %s\n", string);

    // Get device OpenCL version
    err = clGetDeviceInfo(device[j], CL_DEVICE_OPENCL_C_VERSION, sizeof(strin
g), &string, NULL);
    checkError(err, "Getting device OpenCL C version");
    printf("\t\tVersion: %s\n", string);

    // Get Max. Compute units
    cl_uint num;
    err = clGetDeviceInfo(device[j], CL_DEVICE_MAX_COMPUTE_UNITS, sizeof(cl_
uint), &num, NULL);
    checkError(err, "Getting device max compute units");
    printf("\t\tMax. Compute Units: %d\n", num);

    // Get local memory size
    cl_ulong mem_size;
    err = clGetDeviceInfo(device[j], CL_DEVICE_LOCAL_MEM_SIZE, sizeof(cl_ulo
ng), &mem_size, NULL);
    checkError(err, "Getting device local memory size");
    printf("\t\tLocal Memory Size: %llu KB\n", mem_size/1024);

    // Get global memory size
    err = clGetDeviceInfo(device[j], CL_DEVICE_GLOBAL_MEM_SIZE, sizeof(cl_ul
ong), &mem_size, NULL);
    checkError(err, "Getting device global memory size");
    printf("\t\tGlobal Memory Size: %llu MB\n", mem_size/(1024*1024));

    // Get maximum buffer alloc. size
    err = clGetDeviceInfo(device[j], CL_DEVICE_MAX_MEM_ALLOC_SIZE, sizeof(cl
_ulong), &mem_size, NULL);
    checkError(err, "Getting device max allocation size");
    printf("\t\tMax Alloc Size: %llu MB\n", mem_size/(1024*1024));

    // Get work-group size information
    size_t size;
    err = clGetDeviceInfo(device[j], CL_DEVICE_MAX_WORK_GROUP_SIZE, sizeof(s
ize_t), &size, NULL);
    checkError(err, "Getting device max work-group size");
    printf("\t\tMax Work-group Total Size: %ld\n", size);

    // Find the maximum dimensions of the work-groups
    err = clGetDeviceInfo(device[j], CL_DEVICE_MAX_WORK_ITEM_DIMENSIONS, siz
eof(cl_uint), &num, NULL);
    checkError(err, "Getting device max work-item dims");
    // Get the max. dimensions of the work-groups
    size_t dims[num];
    err = clGetDeviceInfo(device[j], CL_DEVICE_MAX_WORK_ITEM_SIZES, sizeof(d
ims), &dims, NULL);
    checkError(err, "Getting device max work-item sizes");
    printf("\t\tMax Work-group Dims: ( ");
    for (size_t k = 0; k < num; k++)
    {
        printf("%ld ", dims[k]);
    }
    printf(")\n");

    printf("\t-----\n");
}

```

```
        printf("\n-----\n");  
    }  
    return EXIT_SUCCESS;  
}
```